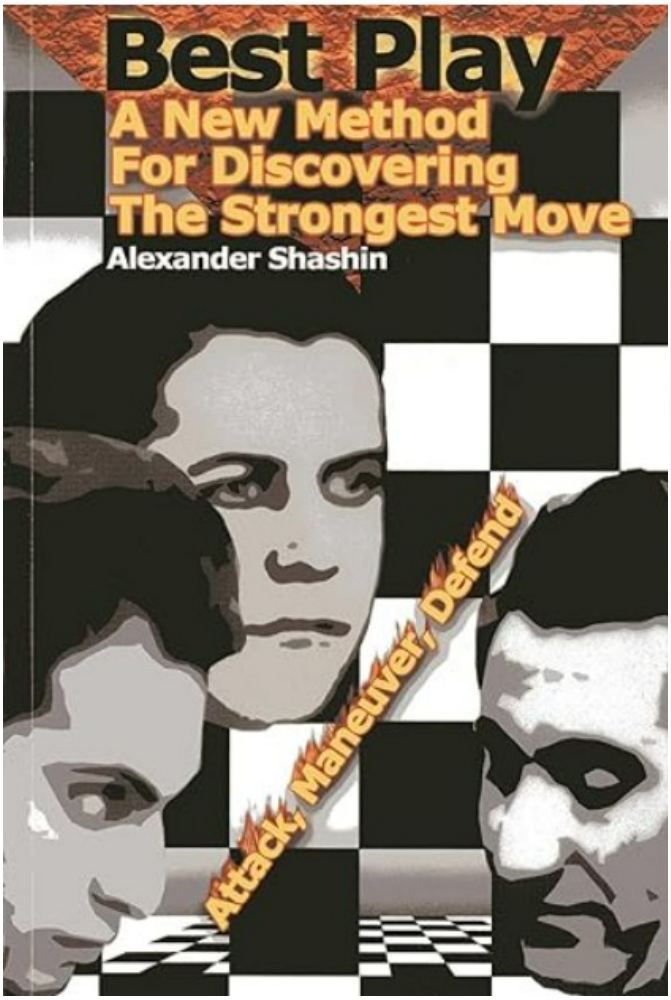
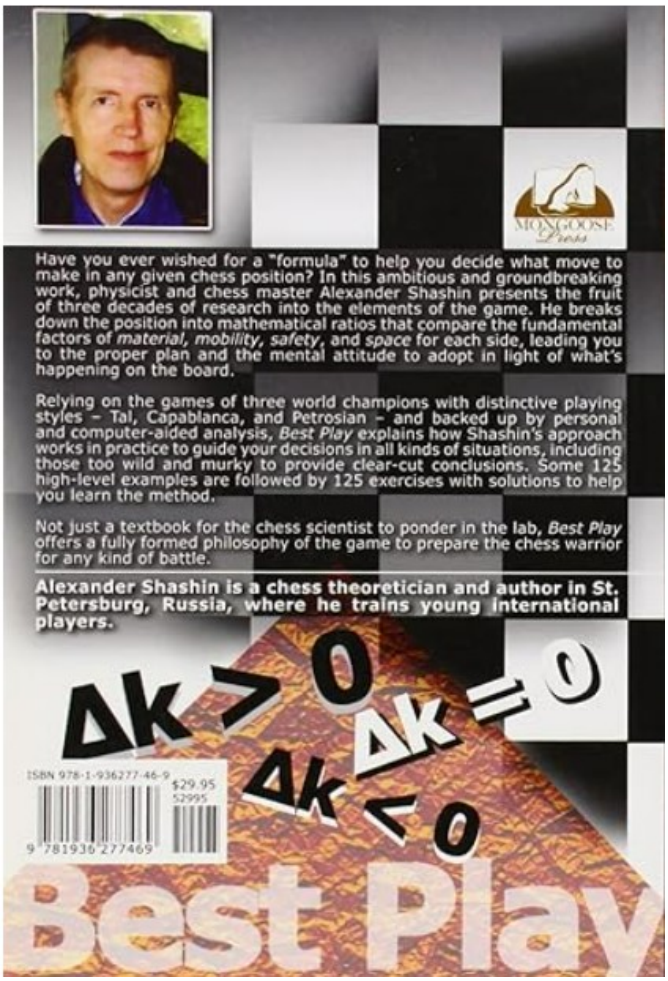
The Shashin Framework: Universal Chess Theory and Algorithmic Implementation

1. Introduction: The Physics of Chess

The framework developed by physicist and chess theorist Alexander Shashin in his book *"Best Play: A New Method for Discovering the Strongest Move"* is not merely a collection of chess principles, but an attempt to apply physical and thermodynamic laws to the chessboard.





The central concept is that a chess game is a dynamic system oscillating between states of equilibrium and chaos. The goal of the player (or the engine) is not to find the "best" move in an abstract sense, but to identify **which algorithm to apply** based on the current state of the system.

Shashin identifies three fundamental algorithms, personified by three World Champions with archetypal styles:

1. **Tal (Attack / Expansion)**
2. **Capablanca (Strategy / Equilibrium)**
3. **Petrosian (Defense / Contraction)**

---

2. The Three Fundamental Algorithms

Shashin decomposes the position into mathematical ratios based on fundamental factors: **Material, Mobility, Safety, and Space**.

A. The Tal Algorithm (Attack)

- **Physical Principle:** Increase in entropy, expansion, release of kinetic energy.
- **Condition:** When the system presents a significant dynamic advantage (space, time/tempi, opponent's king safety compromised).
- **Goal:** Checkmate or decisive material gain.
- **Characteristics:** Material sacrifices ($m > 1$ in favor of the opponent initially, to gain $t > 1$ time), open play, search for forcing lines. In terms of code, this requires a deep search ("Gold Digger") and the disabling of pruning filters to avoid discarding seemingly absurd moves.

B. The Capablanca Algorithm (Strategy)

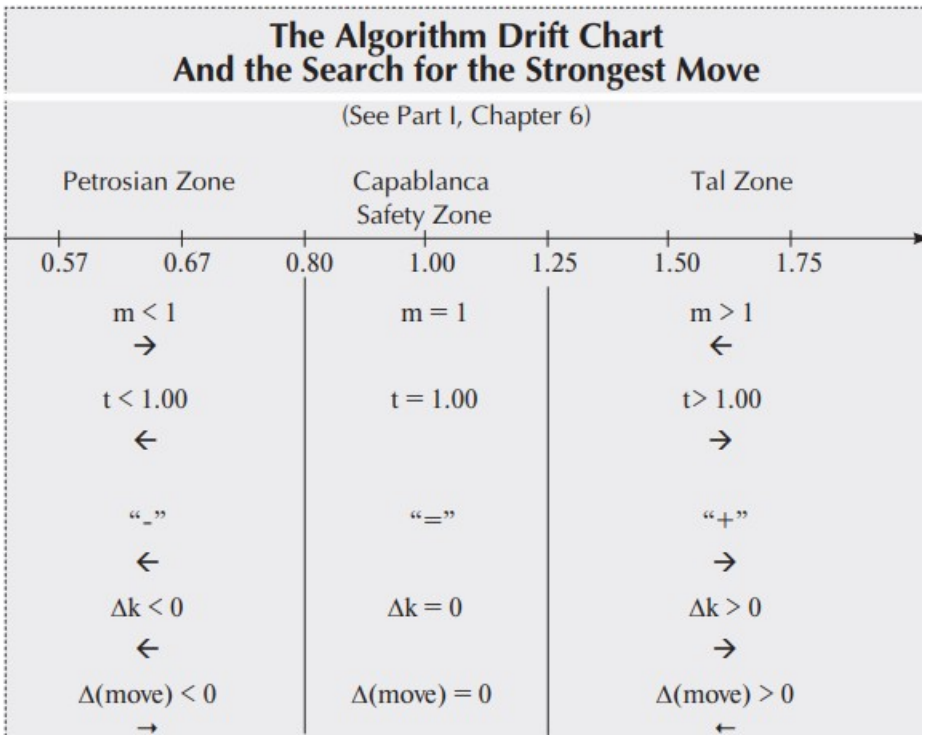
- **Physical Principle:** Conservation of energy, dynamic equilibrium, principle of least action.
- **Condition:** When the position is balanced ($m = 1$, $t = 1$). There are no immediate attacks or lethal threats.
- **Goal:** Improve piece placement (Makogonov Principle), accumulate small advantages, transform static advantage into dynamic advantage.
- **Characteristics:** Prophylaxis, centralization, solid pawn structures. This is the "Safety First" state.

C. The Petrosian Algorithm (Defense)

- **Physical Principle:** Contraction, density, increase in stability to resist external pressure.
- **Condition:** When under attack or at a disadvantage ($t < 1$).
- **Goal:** Neutralize the opponent's initiative, build "fortresses", exchange the opponent's active pieces.
- **Characteristics:** Over-protection, restriction of opponent's mobility, acceptance of passive but solid positions.

---

3. The "Drift Chart" and the Delta Kappa (ΔK) Parameter



The mathematical heart of the system is the **Drift Chart**, which maps every position onto a continuous spectrum.

- **Conceptual Formula:** $K = \frac{Attack + Maneuver}{Defense}$.
- **ΔK (Delta Kappa):** Represents the deviation from equilibrium.
    - **$\Delta K > 0$ (Tal Zone):** The position requires violent attack. The player must look for moves that increase chaos.
    - **$\Delta K = 0$ (Capablanca Zone):** The position is stable. Requires strategic maneuver.
    - **$\Delta K < 0$ (Petrosian Zone):** The position is critical. Requires immediate defense.

The chart defines precise numerical thresholds (e.g., 0.57, 1.00, 1.50) to determine when to switch from one algorithm to another.

---

4. Implementation in ShashChess

ShashChess does not limit itself to using these labels philosophically but translates them into C++ code by modifying the behavior of **Stockfish 18**. Here is how the framework is engineered.

A. WDL Mapping -> Shashin Zones

Instead of manually calculating physical parameters (which would be computationally prohibitive in real-time), ShashChess uses modern NNUE evaluation and the **WDL (Win/Draw/Loss)** model to approximate ΔK.

In the README.md file and the source code, win probability is directly mapped to Shashin Zones:

Win Probability	Shashin Zone	Symbol	Engine Behavior
96-100%	**High Tal**	+-	Total Attack, Pruning disabled
88-95%	**High-Middle Tal**	+/-	Dominance, search for the decisive blow
51-75%	**Chaos: Capablanca-Tal**	↑	Initiative, transition to attack
50%	**Capablanca**	=	Pure strategy, aggressive pruning (efficiency)
25-49%	**Chaos: Capa-Petrosian**	↓	Opponent pressure, prudence

Win Probability	Shashin Zone	Symbol	Engine Behavior
0-15%	**High Petrosian**	-+	Desperate defense, search for stalemate/fortress

B. Search Modifications (search.cpp)

The engine alters its search parameters based on the detected zone:

1. **Aspiration Window:**
    - *Tal/Aggressive:* The window is widened (delta * 9 / 8). The engine expects large score swings (sacrifices) and wants to see them.
    - *Capablanca/Strategical:* The window is standard or reduced. Positional stability is assumed.
2. **LMR (Late Move Reductions):**
    - *Tal:* **Lower reduction (-50/60)**. The engine "trusts less" the static ordering and searches deeper even for moves that look bad (potential brilliant sacrifices).
    - *Capablanca:* **Higher reduction (+45/80)**. If a move doesn't look good immediately, it is discarded quickly to save time and calculate the main lines deeper.
    - *Petrosian:* **Specific reduction (-50)**. In defense, one cannot make mistakes; less pruning is done to avoid missing the only move that saves the game.
3. **Null Move Pruning (NMP):**
    - In *Gold Digger* mode (Extreme Tal), NMP is disabled if the position is "Hot" (NodeHot), meaning if there is danger of checkmate, hanging pieces, or advanced passed pawns. Skipping a move in these situations hides decisive tactics.

C. Modifications to Move Selection (movepick.cpp)

The ordering of moves changes based on the zone:

- **In Capablanca Zone:** A bonus (+30) is given to moves that bring pieces to safe squares (not attacked), favoring solidity.
- **In Tactical Zone:** A bonus (+25) is given to moves that move a piece from a threatened square to a safe one (evasion) or that create direct threats.

D. Optimized Shashin Manager (shashin_manager.cpp)

To make all this efficient, a ShashinManager was introduced that uses advanced caching (Bitmask) to avoid constantly recalculating if a position is a "Fortress" or "Sacrificial". This

allows applying complex theory without slowing down the nodes per second (NPS) calculation.

---

5. The Concept of "Nugget"

The paper *Fewer Draws, More Fun* extends the framework by introducing **Nuggets**.

- **Definition:** A Nugget is an unbalanced starting position, scientifically selected, where the win probability is approximately **75%** (or 25% for Black).
- **Shashin Correspondence:** These positions are often found in transition zones ("Chaos Zones") or "Low Tal", where equilibrium is broken but victory is not trivial.
- **Purpose:** Using these positions in tests prevents engines from taking refuge in "theoretical draws" (Pure Capablanca Zone), forcing them to play in Tal or Petrosian zones where the difference in algorithmic strength emerges clearly.

---

6. Conclusion: A Three-Headed Engine

ShashChess 41 is not simply a stronger clone of Stockfish. It is an engine that **changes personality** dynamically:

1. It becomes a **Tactical Monster** (Tal) when it smells blood.
2. It becomes a **Boa Constrictor** (Petrosian) when it is on the ropes.
3. It becomes a **Positional Grandmaster** (Capablanca) when the position is calm.

This hybrid architecture, guided by Shashin's theory and validated by tests on "Nuggets", explains why ShashChess outperforms Stockfish in long time controls, where understanding the nature of the position prevails over pure brute-force calculation speed.